See Parallel Assignment, on page 148 for examples of parallel assignment. The value of a parallel assignment is its array of rvalues.

## Rightward Assignment, or Single Pattern Matching

*expression* => *pattern*

*expression* in *pattern*

Ruby's pattern matching allows for a single-line form that effectively works as "rightward" assignment, where the existing value is on the left and the variables to be assigned are on the right.

```
3 => x
puts x
```

```
{a: 1, b: 2} in {a:, b:}
puts a
puts b
```

*produces:*

```
3
1
2
```

The difference between the two forms is what happens if the pattern and the expression don't match.

The => form returns nil on a successful match and throws a NoMatchingPatternKeyError if it can not match the pattern. The in form returns true if the pattern matches and false if the pattern does not match. In the not-matching case, the behavior of the variables that would have been matched is undefined—if the match returns false you cannot depend on those values being matched, unmatched, or anything else.

For the exact syntax of patterns, see Case Pattern Matching, on page 460—the definition of a pattern is the same, and we'll talk more about the semantics of patterns there.

## Block Expressions

```
begin
  body
end
```

Expressions may be grouped between begin and end. The value of the block expression is the value of the last expression executed.

Block expressions also play a role in exception handling—see Exceptions, on page 483.

## Boolean Expressions

Ruby predefines the constants false and nil. Both of these values are treated as being false in a boolean context. All other values are treated as being true. The constant true is available for when you need an explicit "true" value.